

Estructura de datos

Listas circulares

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

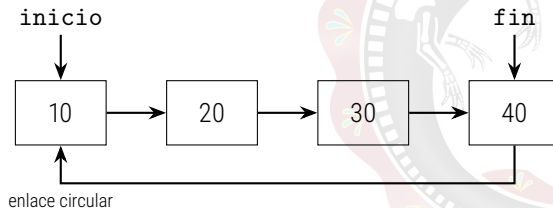
- 1 Listas circulares
- 2 Insertar nodos
- 3 Métodos auxiliares `obtenerNodo()` y `obtenerIndice()`
- 4 Buscar nodos
- 5 Eliminar nodos
- 6 Métodos `tamano()` y `estaVacia()`
- 7 Limpiar lista



Lista circular

Definición

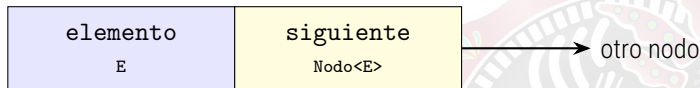
Una **lista simplemente enlazada circular** es una lista en la que la **referencia siguiente** del **último nodo** **apunta** nuevamente al **primer nodo**.



¡Importante!

El último nodo **no apunta a null**, apunta a **inicio**

Clase Nodo<E>



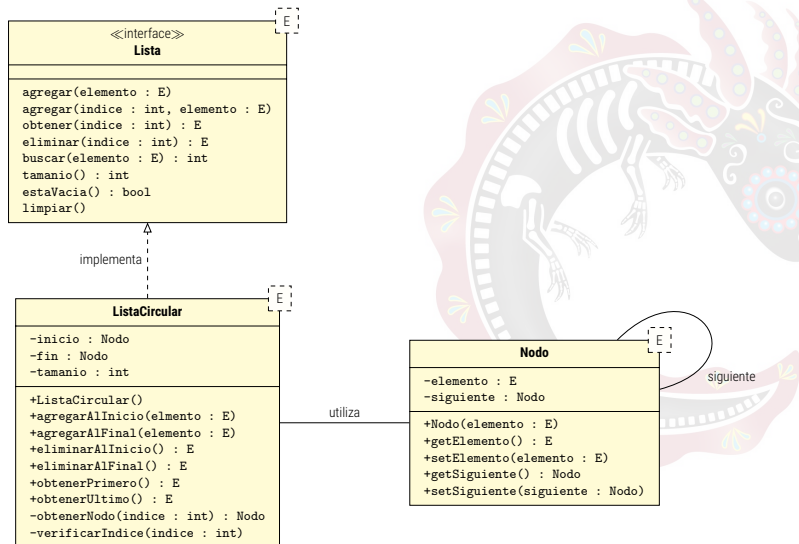
Reutilización

`ListaCircular<E>` utiliza **la misma clase** `Nodo<E>` que `ListaEnlazada<E>`. La diferencia está en la forma en que se **organizan las referencias**.

Lista simple: 30 → null

Lista circular: 30 → inicio

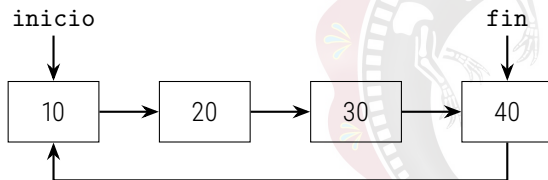
Diagrama de clases



Representación interna

`ListaCircular<E>` mantiene **tres atributos**:

- **inicio**: referencia al **primer nodo**.
- **fin**: referencia al **último nodo**.
- **tamaño**: número de **elementos almacenados**.



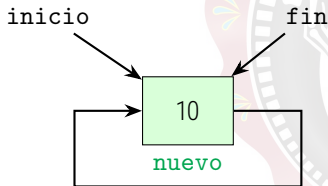
Invariantes

- **Lista vacía**: `inicio == null` y `fin == null`.
- **Lista no vacía**: `fin.getSiguiente() == inicio`.
- **tamaño** coincide con el **número de nodos**.

El primer nodo de la lista

Al agregar el **primer elemento**:

- 1: $nuevo \leftarrow \text{Nodo}(10)$
- 2: $inicio \leftarrow nuevo$
- 3: $fin \leftarrow nuevo$
- 4: $nuevo.siguiente \leftarrow nuevo$



¡Importante!

Con un **único elemento**: $inicio == fin$ e $inicio.getSiguiente() == inicio$

Método agregar(elemento)

Require: elemento

```

1: nuevo  $\leftarrow$  Nodo(elemento)
2: if la lista está vacía then
3:   inicio  $\leftarrow$  nuevo
4:   fin  $\leftarrow$  nuevo
5:   nuevo.siguiente  $\leftarrow$  nuevo
6: else
7:   nuevo.siguiente  $\leftarrow$  inicio
8:   fin.siguiente  $\leftarrow$  nuevo
9:   fin  $\leftarrow$  nuevo
10: end if
11: tamano  $\leftarrow$  tamano + 1

```



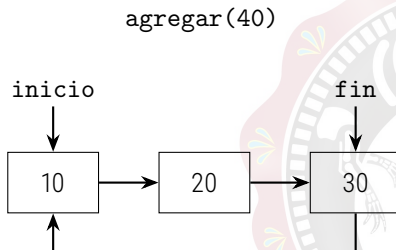
¡Importante!

Gracias a la referencia **fin**, **agregar al final** no requiere recorrer la lista:

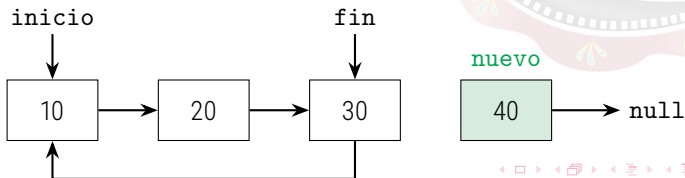
$$O(1)$$

Agregar un elemento al final

Ejemplo:

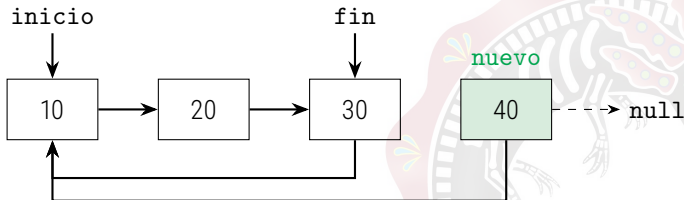


Creamos un nodo **nuevo**:

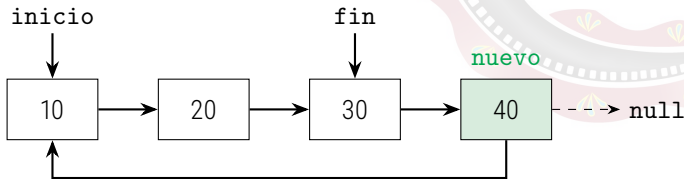


Agregar un elemento al final

Cambiamos la referencia `nuevo.siguiente = inicio`

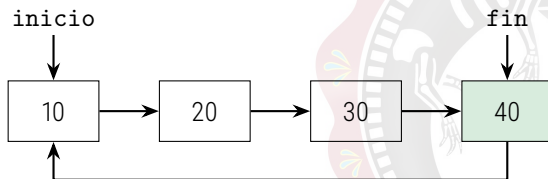


Cambiamos la referencia `fin.siguiente = nuevo`



Agregar un elemento al final

Finalmente actualizamos `fin = nuevo`



`tamano = 4`

Insertión al inicio

Para insertar en la posición 0:

- 1: $nuevo \leftarrow \text{Nodo}(\text{elemento})$
- 2: $nuevo.siguiete \leftarrow inicio$
- 3: $inicio \leftarrow nuevo$
- 4: $fin.siguiete \leftarrow inicio$
- 5: $tamano \leftarrow tamano + 1$

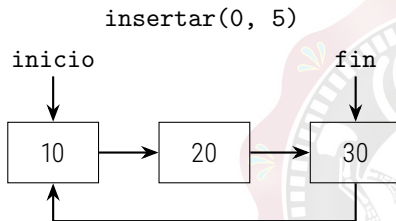


¡Importante!

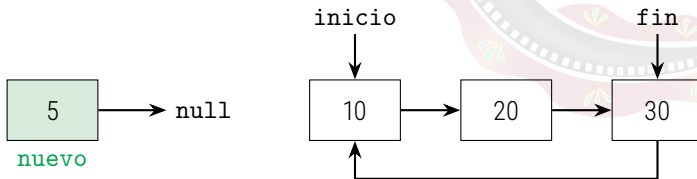
Al cambiar **inicio**, también **debemos modificar fin.siguiete** para **conservar la circularidad**.

Inserción al inicio

Ejemplo:

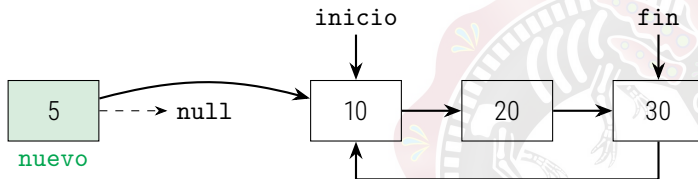


Creamos un nodo **nuevo**

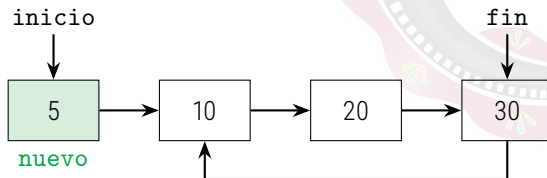


Inserción al inicio

Enlazamos `nuevo.siguiente = inicio`

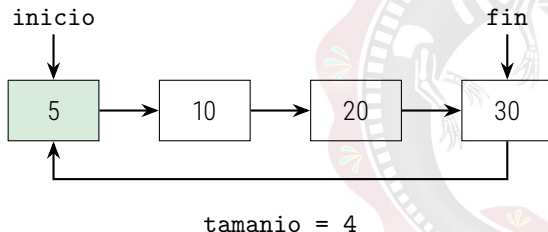


Actualizamos `inicio = nuevo`



Inserción al inicio

Finalmente, actualizamos `fin.siguiente = inicio`



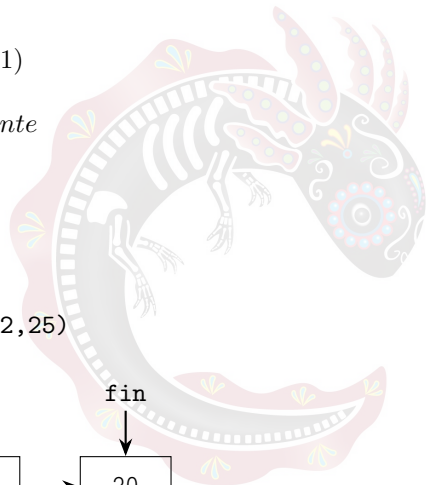
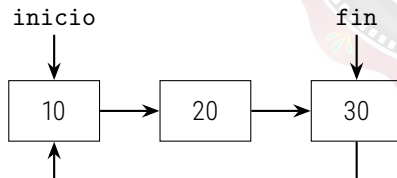
Insertión intermedia

Para insertar en una posición diferente de 0:

- 1: $\text{anterior} \leftarrow \text{obtenerNodo}(\text{indice} - 1)$
- 2: $\text{nuevo} \leftarrow \text{Nodo}(\text{elemento})$
- 3: $\text{nuevo.siguiente} \leftarrow \text{anterior.siguiente}$
- 4: $\text{anterior.siguiente} \leftarrow \text{nuevo}$
- 5: $\text{tamanio} \leftarrow \text{tamanio} + 1$

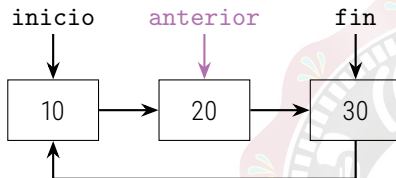
Ejemplo:

insertar(2,25)

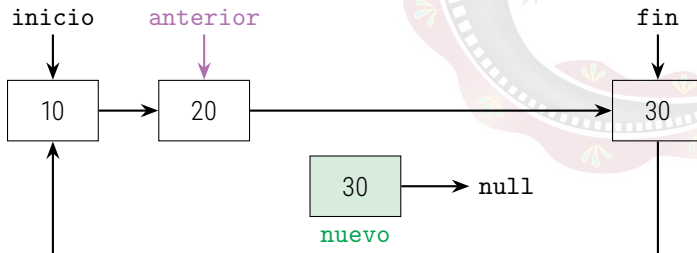


Insertión intermedia

Obtenemos la referencia al nodo **anterior** a la posición a insertar

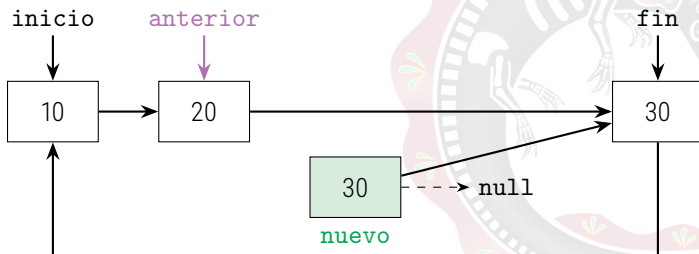


Creamos un nodo **nuevo**



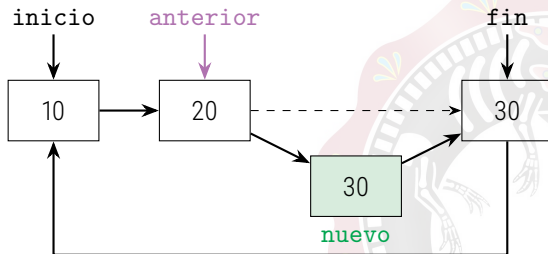
Inserción intermedia

Actualizamos la referencia `nuevo.siguiiente = anterior.siguiiente`

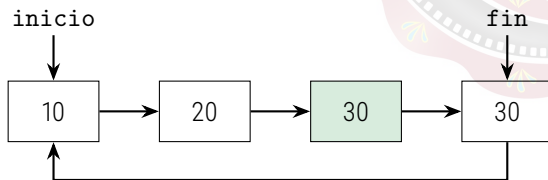


Inserción intermedia

Actualizamos `anterior.siguiente = nuevo`



Finalmente

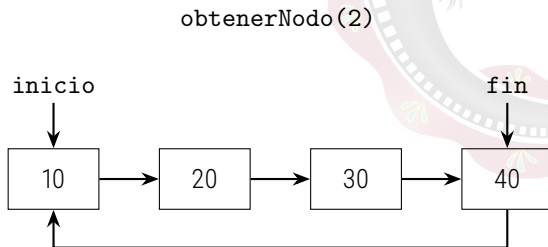


Método `obtenerNodo(indice)`

Require: `indice`

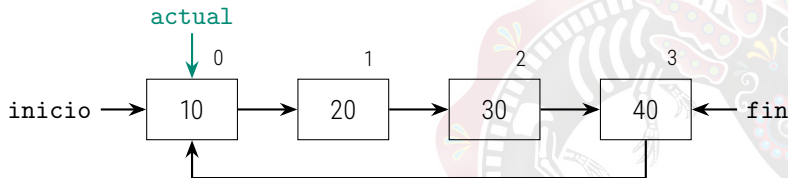
- 1: $actual \leftarrow inicio$
- 2: **for** $i \leftarrow 0$ **hasta** $indice - 1$ **do**
- 3: $actual \leftarrow actual.siguiete$
- 4: **end for**
- 5: **return** $actual$

Ejemplo

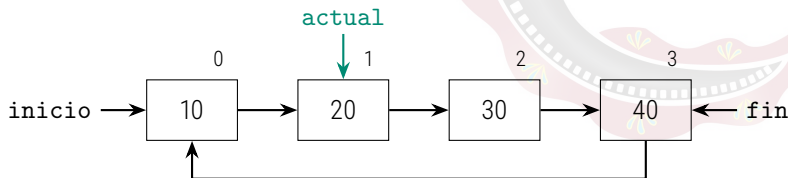


Método `obtenerNodo(indice)`

Apuntamos `actual = inicio`

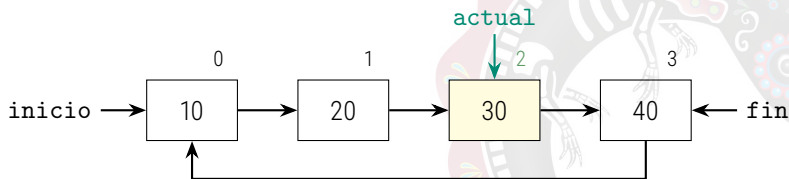


Recorreremos



Método `obtenerNode(indice)`

Cuando $i = 2$ finalizamos el ciclo



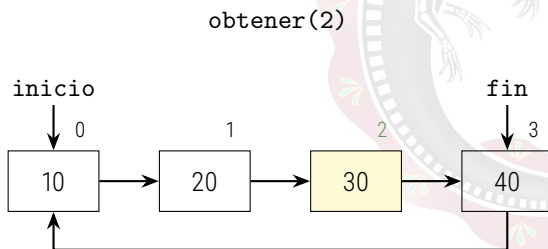
`obtenerNode(2) → Nodo(30)`

Método `obtener(indice)`

Require: indice

- 1: `verificarIndice(indice)`
- 2: $nodo \leftarrow obtenerNodo(indice)$
- 3: **return** `nodo.elemento`

Ejemplo:



`obtener(2) = 30`

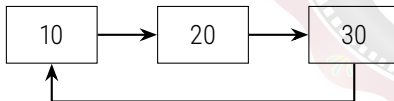
Recorrido de una lista circular

En una **lista simplemente enlazada** utilizábamos:

```
while (actual != null) {
    actual = actual.getSiguiente();
}
```

¡Importante!

actual **nunca** llegará a **null**. Después del último nodo se **regresa nuevamente a inicio**.

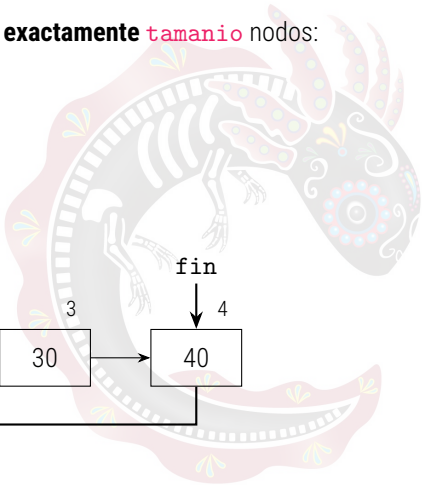


10 → 20 → 30 → 10 → 20 → 30 → ...

```

1: actual  $\leftarrow$  inicio
2: for i  $\leftarrow$  0 hasta tamano - 1 do
3:   procesar actual.elemento
4:   actual  $\leftarrow$  actual.siguiete
5: end for

```



El recorrido **termina** por el **número de nodos procesados**, **no** porque se encuentre una **referencia null**.

Método buscar(elemento)

Require: elemento

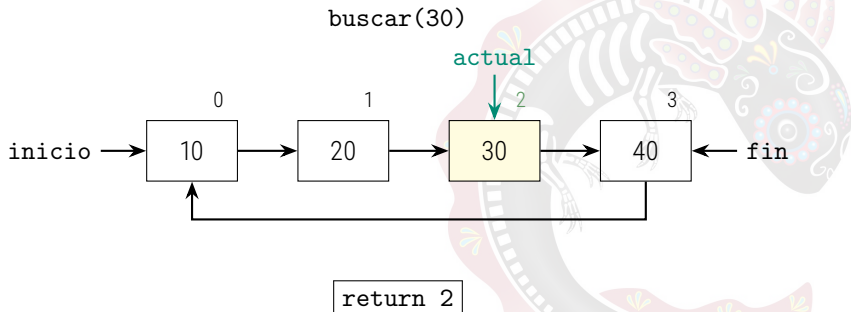
```

1: if la lista está vacía then
2:     return -1
3: end if
4: actual  $\leftarrow$  inicio
5: for indice  $\leftarrow$  0 hasta tamaño - 1 do
6:     if actual.elemento = elemento then
7:         return indice
8:     end if
9:     actual  $\leftarrow$  actual.siguiente
10: end for
11: return -1
    
```



Ejemplo de búsqueda

Ejemplo:



Complejidad

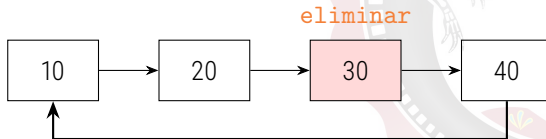
En el peor caso se recorren todos los nodos:

$O(n)$

Eliminación de elementos

Para **eliminar** un elemento debemos **considerar**:

- 1 La lista contiene un **único nodo**.
- 2 Se elimina el **primer nodo**.
- 3 Se elimina un **nodo intermedio**.
- 4 Se elimina el **último nodo**.



¡Importante!

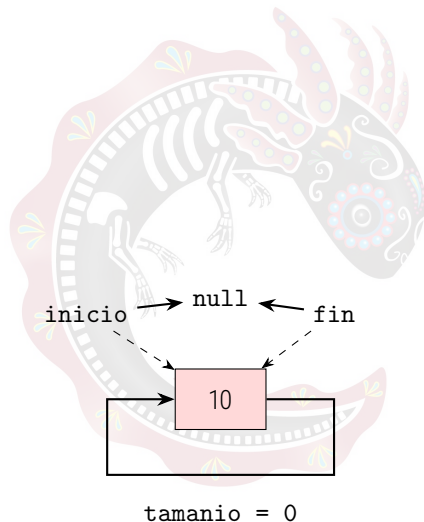
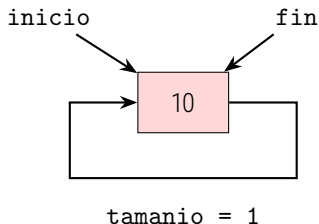
Después de **eliminar el nodo** debe conservarse `fin.siguiente == inicio`

Eliminar el único elemento

```

1: if tamano = 1 then
2:   eliminado  $\leftarrow$  inicio
3:   inicio  $\leftarrow$  null
4:   fin  $\leftarrow$  null
5:   tamano  $\leftarrow$  0
6: end if

```

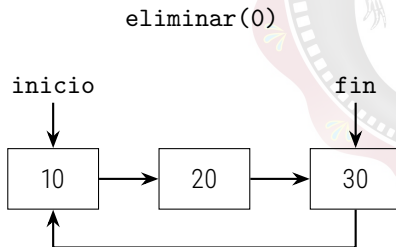


Eliminar el primer nodo

Si `indice == 0`:

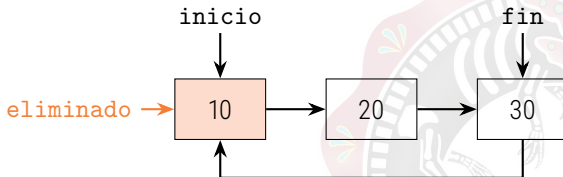
- 1: $\text{eliminado} \leftarrow \text{inicio}$
- 2: $\text{inicio} \leftarrow \text{inicio.siguiente}$
- 3: $\text{fin.siguiente} \leftarrow \text{inicio}$
- 4: $\text{tamanio} \leftarrow \text{tamanio} - 1$

Ejemplo

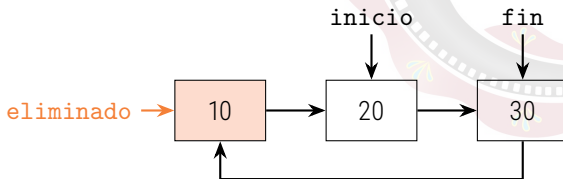


Eliminar el primer nodo

Apuntamos el nodo `inicio` como `eliminado`

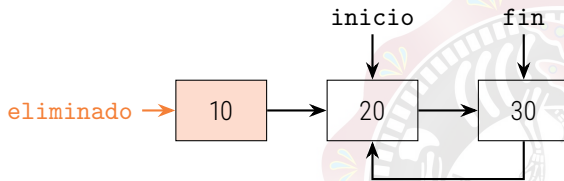


Cambiamos la referencia de `inicio = inicio.siguiente`

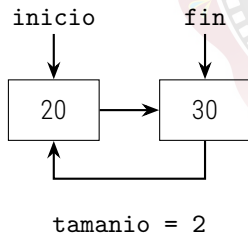


Eliminar el primer nodo

Cambiamos la referencia `fin.siguiente = inicio`



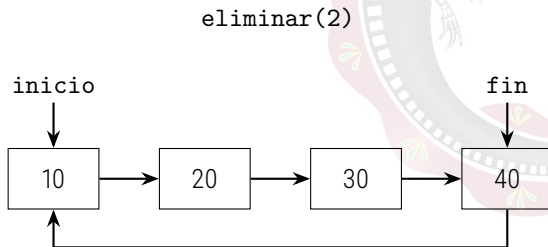
Finalmente

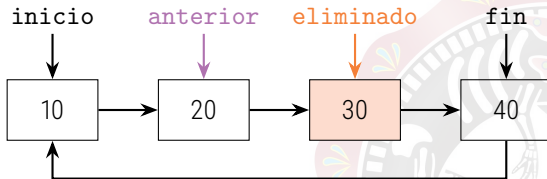


Eliminación intermedia

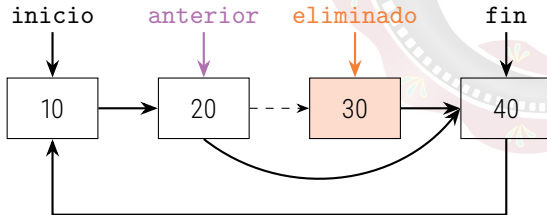
- 1: $\text{anterior} \leftarrow \text{obtenerNodo}(\text{indice} - 1)$
- 2: $\text{eliminado} \leftarrow \text{anterior.siguiente}$
- 3: $\text{anterior.siguiente} \leftarrow \text{eliminado.siguiente}$
- 4: $\text{tamanio} \leftarrow \text{tamanio} - 1$

Ejemplo:



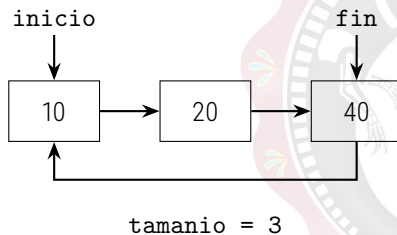


Cambiamos la referencia `anterior.siguiente = eliminado.siguiente`



Eliminación intermedia

Finalmente

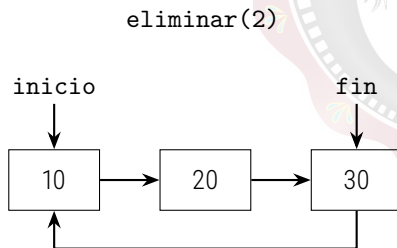


Eliminar el último nodo

Si el nodo eliminado es **fin**:

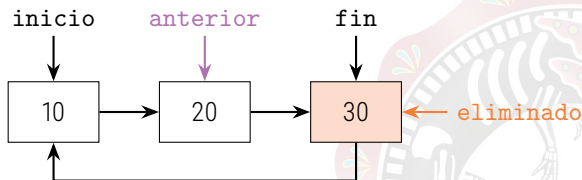
- 1: $\text{anterior} \leftarrow \text{obtenerNodo}(\text{indice} - 1)$
- 2: $\text{eliminado} \leftarrow \text{anterior.siguiente}$
- 3: $\text{anterior.siguiente} \leftarrow \text{eliminado.siguiente}$
- 4: $\text{fin} \leftarrow \text{anterior}$
- 5: $\text{tamanio} \leftarrow \text{tamanio} - 1$

Ejemplo:

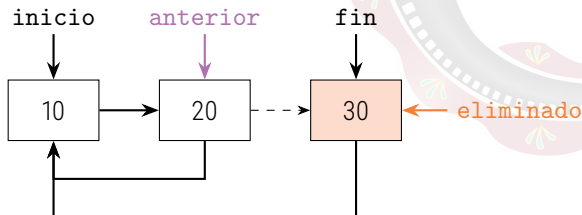


Eliminar el último nodo

Marcamos el nodo **anterior** al **eliminado**

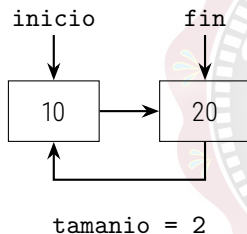


Cambiamos la referencia **anterior.siguiente** = **eliminado.siguiente**



Eliminar el último nodo

Finalmente `fin = anterior`



Método `eliminar(indice)`

Require: `indice`

```

1: verificarIndice(indice)
2: if tamaño = 1 then
3:   eliminado ← inicio
4:   inicio ← null
5:   fin ← null
6: else if indice = 0 then
7:   eliminado ← inicio
8:   inicio ← inicio.siguiente
9:   fin.siguiente ← inicio
10: else
11:   anterior ← obtenerNodo(indice - 1)
12:   eliminado ← anterior.siguiente
13:   anterior.siguiente ← eliminado.siguiente
14:   if eliminado = fin then
15:     fin ← anterior
16:   end if
17: end if
18: tamaño ← tamaño - 1
19: return eliminado.elemento

```



Métodos `tamano()` y `estaVacia()`

`tamano()`

return *tamano*

10 → 20 → 30 ↻

`tamano() = 3`

`estaVacia()`

return *tamano* = 0

También puede verificar

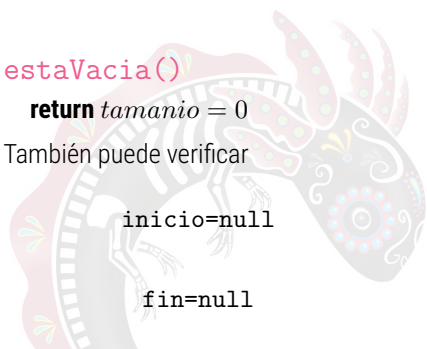
`inicio=null`

`fin=null`

¡Importante!

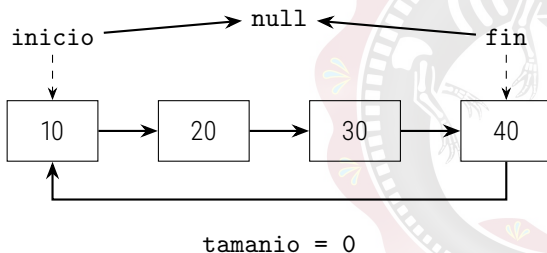
Ambas operaciones se realizan en:

$O(1)$



Método limpiar()

- 1: $inicio \leftarrow null$
- 2: $fin \leftarrow null$
- 3: $tamaño \leftarrow 0$



Administración de memoria

Al **eliminar las referencias** externas a los nodos, la estructura **deja de ser accesible** desde la lista y puede ser recuperada posteriormente por el **recolector de basura** de Java ☕.

Invariantes de la estructura

Después de cada operación, una `ListaCircular<E>` válida **debe cumplir**:

1 Si `tamaño == 0`:

```
inicio == null    fin == null
```

2 Si `tamaño > 0`:

```
fin.getSiguiente() == inicio
```

3 Si `tamaño == 1`:

```
inicio == fin
```

4 Partiendo de `inicio` y avanzando `tamaño` veces:

```
actual == inicio
```

¡Importante!

Estas **propiedades** permiten **verificar** si una **inserción o eliminación** ha dejado la estructura en un **estado válido**.

¿Por qué conservar *fin*?

Consideremos la operación:

agregar(elemento)

Sin referencia a *fin*

Es necesario recorrer:

inicio $\rightarrow \dots \rightarrow$ *ultimo*

$O(n)$

Con referencia a *fin*

Accedemos directamente:

fin.siguiente = nuevo

$O(1)$

¡Importante!

Mantener *fin* requiere **actualizar esta referencia** correctamente durante las inserciones y eliminaciones.

Comparación de estructuras

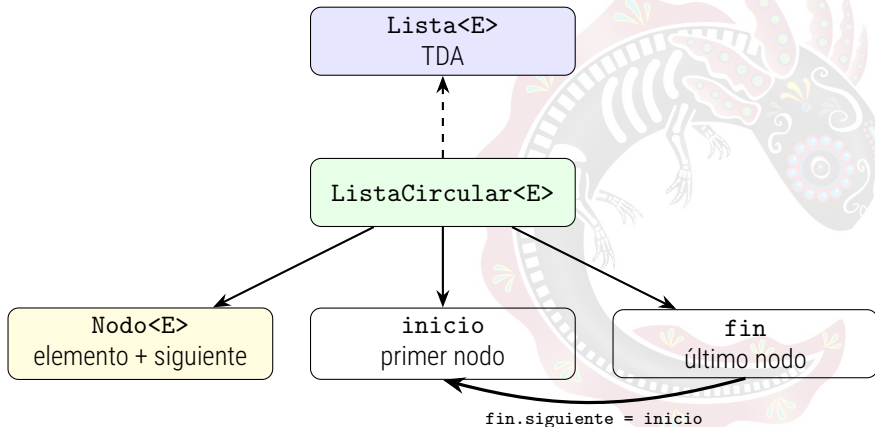


Característica	ListaEnlazada	ListaDoblementeEnlazada	ListaCircular
Referencias por nodo	1	2	1
Referencia a inicio	Sí	Sí	Sí
Referencia a fin	No	Sí	Sí
Último nodo apunta a	null	null	inicio
Recorrido hacia atrás	No	Sí	No
Recorrido cíclico	No	No	Sí
Agregar al final	$O(n)$	$O(1)$	$O(1)$

Complejidad temporal

Operación	Complejidad
agregar(elemento)	$O(1)$
agregar(indice, elemento)	$O(n)$
obtener(indice)	$O(n)$
eliminar(indice)	$O(n)$
buscar(elemento)	$O(n)$
tamano()	$O(1)$
estaVacía()	$O(1)$
limpiar()	$O(1)$

Implementación de ListaCircular<E>



Idea fundamental

La implementación es similar a una lista simplemente enlazada, pero debe mantenerse en todo momento la **conexión entre el último y el primer nodo**.

Fin de la presentación

¿Preguntas o comentarios?

